

INFORME TÉCNICO

Investigación y Consumo de API Externa



PayPal REST API+ Proyecto ServiAT

Instructor: Ing. Edwin Marín | 12 de mayo del 2026

Aprendiz:

Santiago De hoyos

Nikol Tatiana Ortiz

Danna Valentina Ortiz

Nicolás rojas rocha

1. API SELECCIONADA: PayPal REST API

1.1 Justificación de la elección

Elegí la API de PayPal porque permite realizar pagos en línea de forma segura y rápida, sin necesidad de crear un sistema de pagos desde cero. Esta API es muy útil en aplicaciones modernas, ya que facilita la integración de servicios financieros dentro de una plataforma.

Necesidad en tu proyecto	Solución con api externa
En mi caso, esta API se relaciona directamente con mi sistema de información, ya que estamos desarrollando un sistema donde se generan tickets para los usuarios.	Por lo tanto, es importante contar con una forma de pago confiable que permita a los usuarios realizar sus transacciones de manera sencilla.

Necesidad en tu proyecto	Solución con api externa

1.2 Problema que resuelve la API

R// Implementar esta Api en mi proyecto nos ayuda a:

1. Procesar pagos de forma segura.
Sin PayPal tendríamos que:
2. Guardar o procesar tarjetas (muy peligroso y complejo)
3. Cumplir normas de seguridad (PCI-DSS)
Con PayPal:
4. El usuario paga en PayPal (no en tu sistema)
5. Tú nunca ves los datos de la tarjeta

2. INVESTIGACIÓN TÉCNICA DE LA API

2.1 Documentación oficial

URL de documentación: <https://developer.paypal.com/docs/api/overview/>

La documentación oficial de PayPal permite entender cómo funciona la API, los endpoints disponibles, los métodos HTTP y la forma de autenticación mediante OAuth 2.0.

2.2 Métodos HTTP y endpoints clave

Método	Endpoint	Uso en	Respuesta clave
POST	https://api-m.sandbox.paypal.com/v1/oauth2/token	Obtener token de acceso (autenticación)	200 OK (access_token)
POST	https://api-m.sandbox.paypal.com/v2/checkout/orders	Crear una orden de pago	201 Created (orden creada)
GET	https://api-m.sandbox.paypal.com/v2/checkout/orders/35G32488FA498202C	Capturar estado de la orden	200 Created (orden creada)
POST	https://api-m.sandbox.paypal.com/v2/checkout/orders/35G32488FA498202C/capture	Finalizar pago/captura	200 Created (Orden creada)

2.3 Autenticación

La API de PayPal utiliza un sistema de autenticación basado en OAuth 2.0, el cual fue el método implementado durante la práctica en Postman.

En el desarrollo realizado, primero se hizo una petición tipo POST al endpoint `/v1/oauth2/token`, utilizando Basic Auth con el Client ID y el Secret obtenidos desde la plataforma de PayPal. En el Body se envió el parámetro `grant_type=client_credentials`, lo que permitió generar un `access_token`.

Una vez obtenido el token, este se utilizó en las siguientes peticiones, específicamente en la creación de órdenes de pago, mediante el tipo de autorización Bearer Token en Postman. Esto permitió autenticar correctamente las solicitudes y acceder a los endpoints protegidos de la API.

2.4 Límites de uso y políticas

Aspecto	Detalle
Entorno gratuito (Sandbox)	PayPal ofrece un entorno de pruebas llamado Sandbox , donde se pueden realizar pagos simulados sin usar dinero real. Es ideal para desarrollo y pruebas.
Costos en producción	PayPal no cobra por usar la API, pero sí aplica comisiones por cada transacción realizada en producción (pagos reales)
Límites de uso	No hay un límite fijo como otras APIs, pero PayPal monitorea el uso y puede restringir solicitudes si detecta actividad sospechosa o abuso.
Seguridad	No se deben compartir el Client ID, Secret ni el <code>access_token</code> , ya que pueden ser usados por terceros para realizar operaciones.
Políticas de uso	Se debe cumplir con las políticas de PayPal, como no usar la API para actividades fraudulentas o ilegales.

Aspecto	Detalle
Expiración del token	El access_token tiene un tiempo limitado de uso (expira), por lo que se debe generar uno nuevo cuando sea necesario.

3. IMPLEMENTACIÓN TÉCNICA

3.1 Estructura del proyecto de demostración ejemplo

En el proyecto se implementó la API de PayPal mediante endpoints POST y GET probados en Postman. El sistema permite crear órdenes, consultar pagos y capturar transacciones desde el frontend desarrollado en React (`Pago.jsx`) y un backend en Node.js.

The screenshot shows a VS Code editor with a project named 'serviAT'. The Explorer panel on the left shows the project structure:

- serviAT
 - node_modules
 - public
 - src
 - assets
 - components
 - acercaNosotros.jsx
 - Cards.jsx
 - footer.jsx
 - historialVersiones.jsx
 - index.jsx
 - infoFormulario.jsx
 - login.css
 - login.jsx
 - navVar.jsx
 - pago.jsx
 - registro.css
 - registro.jsx
 - servicioConfirmado.jsx
 - serviciosNav.jsx
 - solicitudFormulario.jsx
 - steppers.jsx
 - Tabla.jsx
 - validarServicio.jsx
 - pages
 - supabase
 - App.css
 - App.jsx
 - index.css
 - main.jsx
 - .env.local
 - .gitignore
 - eslint.config.js
 - index.html
 - package-lock.json
 - package.json
 - README.md
 - server.js
 - vite.config.js

The main editor shows the `server.js` file with the following code:

```

1 import express from 'express';
2 import cors from 'cors';
3
4 const app = express();
5
6
7 app.use(cors({
8   origin: 'http://localhost:5173',
9   methods: ['GET', 'POST', 'PUT', 'DELETE'],
10  allowedHeaders: ['Content-Type', 'Authorization']
11 }));
12
13 app.get('/api/servicios', (req, res) => {
14   res.json({ mensaje: "CORS configurado exitosamente" });
15 });
16
17 app.listen(3000, () => console.log("Servidor con CORS en puerto 3000"));

```

The bottom panel shows the terminal with the following output:

```

PS C:\Users\Aprendiz\Downloads\serviAT> npm run dev
VITE v8.0.10 ready in 237 ms
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help

```

3.2 Consumo funcional — Creación de pago (POST)

Ejemplo: Este endpoint POST permite crear una orden de pago utilizando la API de PayPal. El usuario inicia el proceso desde el componente `Pago.jsx`, y el backend genera una orden que luego será aprobada en PayPal.

- Recibe la solicitud del frontend (`Pago.jsx`)
- Crea una orden en PayPal
- Devuelve el enlace de aprobación del pago
- Fue probado correctamente en Postman

Imagen:

```
50 <div className="mt-2">
51   <PayPalButtons
52     style={{ layout: "vertical", shape: "rect", color: "blue" }}
53     createOrder={(data, actions) => {
54
55       setAlerta({ tipo: "", mensaje: "" });
56
57       return actions.order.create({
58         purchase_units: [
59           {
60             amount: {
61               value: "10.00",
62             },
63           },
64         ],
65       });
66     }}
```

3.3 Consumo GET — Consultar estado de una orden

Este endpoint GET permite consultar el estado de un pago realizado en PayPal.

Este endpoint permite:

- Verificar si el pago fue aprobado
- Consultar el estado de la transacción
- Mostrar información del pago en tiempo real

Imagen:

```
</> JSON

{
  "id": "5G20918066888201F",
  "status": "APPROVED",
  "intent": "CAPTURE"
}
```

3.4 Captura del pago (POST)

Después de que el usuario aprueba el pago en PayPal, el backend captura la transacción.

```
</> JavaScript

setAlerta({
  tipo: "success",
  mensaje: `¡Pago exitoso! Gracias por confiar en nosotros`
});
```

4. SEGURIDAD Y BUENAS PRÁCTICAS

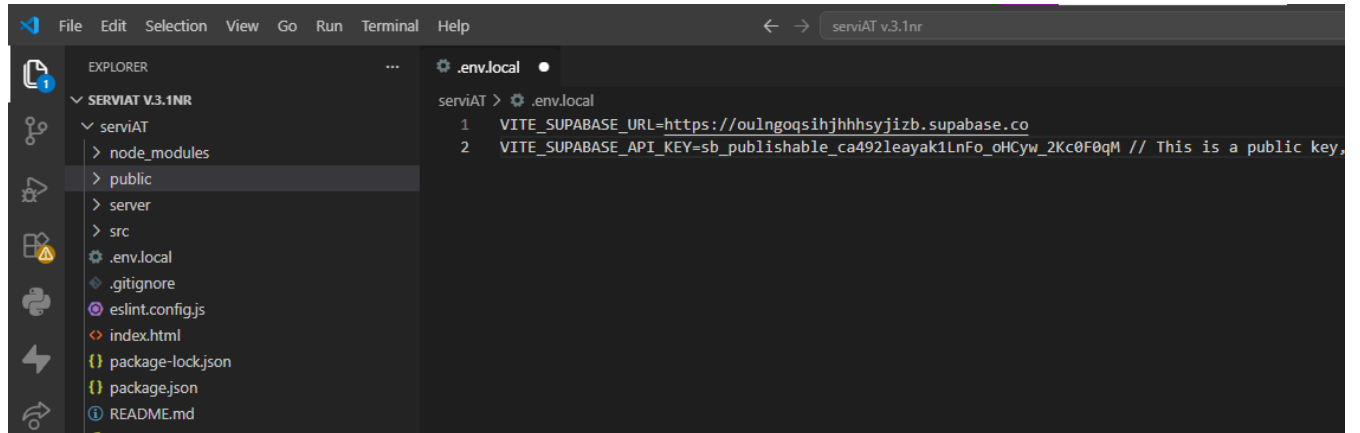
4.1 Protección de credenciales

Las credenciales de PayPal se almacenan en variables de entorno para evitar exponer información sensible.

REST API apps

Important: Store credentials safely, and never share client IDs or secrets publicly.

App name ↑	Client ID	Secret	Date and time ↑	
Default Application	AfTDmIB66lOQ98e099-b2...		3/05/2026, 7:42 PM	



4.2 Riesgos de seguridad identificados

Riesgo	Mitigación aplicada
Exposición de credenciales	Uso de .env
Acceso no autorizado a pagos	Uso de tokens Bearer
Errores de autenticación	Manejo de errores HTTP
Filtración de información	No subir claves a GitHub

5. PREGUNTAS DE INVESTIGACIÓN

P1. ¿Qué problema resuelve la API de PayPal?

La API de PayPal permite integrar pagos electrónicos seguros en aplicaciones web sin necesidad de manejar directamente información bancaria o tarjetas de crédito.

P2. Métodos HTTP y datos que retorna:

POST: Sirve para generar el token: <https://api-m.sandbox.paypal.com/v1/oauth2/token>

POST: Sirve para crear una orden: <https://api-m.sandbox.paypal.com/v2/checkout/orders>.

GET: Sirve para consultar el estado de la orden: <https://api-m.sandbox.paypal.com/v2/checkout/orders/>

POST: Sirve para confirmar el pago:
<https://api-m.sandbox.paypal.com/v2/checkout/orders/35G32488FA498202C/capture>

Datos retornados:

- ID de la orden

- Estado del pago
- Monto
- Enlaces de aprobación

P3. ¿Cómo se autentica?

La API de PayPal utiliza autenticación OAuth 2.0 mediante:

- CLIENT_ID
- SECRET
- Token Bearer

P4. Límites gratuitos y políticas de uso

PayPal Developer ofrece un entorno Sandbox gratuito para pruebas de integración sin utilizar dinero real.

P5. Riesgos de seguridad

Los principales riesgos son:

- Robo de credenciales
- Acceso no autorizado
- Manipulación de pagos
- Exposición de tokens

Por esta razón se utilizan variables de entorno y autenticación segura.

6. EVIDENCIAS DE PRUEBA

6.1 Pruebas locales — realiza las pruebas y toma captura de pantalla toma como ejemplo el siguiente

6.2 Servidor corriendo en localhost:5175


```
serviAT v3.1nr
1 const express = require('express');
2 const cors = require('cors');
3 const app = express();
4
5
6 app.use(cors({
7   origin: 'http://localhost:5175',
8   methods: ['GET', 'POST', 'PUT', 'DELETE'],
9   allowedHeaders: ['Content-Type', 'Authorization']
10 }));
11
12 app.get('/api/servicios', (req, res) => {
13   res.json({ mensaje: "CORS configurado exitosamente" });
14 });
15
16 app.listen(3000, () => console.log("Servidor con CORS en puerto 3000"));

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\nanit\Downloads\serviAT v3.1nr\serviAT> npm run dev

VITE v8.0.10 ready in 1782 ms
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

6.3 Prueba exitosa —Generar token(Post)

POST

https://api-m.sandbox.paypal.com/v1/oauth2/token

Send

Docs Params Authorization Headers 10 Body Scripts Settings

Auth Type

Basic A...

The authorization header will be automatically generated when you send the request. Learn more about [Basic Auth](#) authorization.

Username

AFTDmIB66IOQ98e099-b2oIKUQSTrP5xeFt...

Password

.....

Body Cookies 6 Headers 22 Test Results

200 OK • 1.23 s • 1.83 KB

Save Response

{ } JSON

Preview Visualize

```
1 {
2   "scope": "https://uri.paypal.com/services/invoicing https://uri.paypal.com/services/checkout/one-click-with-merchant-issued-token https://uri.paypal.com/services/payments/futurepayments https://uri.paypal.com/services/vault/payment-tokens/read https://uri.paypal.com/services/disputes/read-buyer https://uri.paypal.com/services/payments/realtimepayment https://uri.paypal.com/services/subscriptions/confirm-payment https://uri.paypal.com/services/payments/client-payments-eligibility https://api.paypal.com/v1/vault/credit-card https://api.paypal.com/v1/payments/* https://uri.paypal.com/services/vault/payment-tokens/readwrite https://uri.paypal.com/services/applications/webhooks https://uri.paypal.com/services/pricing/exchange-currency/read https://uri.paypal.com/services/credit/client-offer-presentment/read https://uri.paypal.com/services/disputes/update-seller https://uri.paypal.com/services/payments/payment/authcapture openid Braintree:Vault https://uri.paypal.com/services/disputes/read-seller https://uri.paypal.com/services/payments/orders/client_sdk_orders_api https://uri.paypal.com/services/payments/refund https://uri.paypal.com/services/disputes/create https://uri.paypal.com/services/pricing/quote-exchange-rates/read https://uri.paypal.com/services/documents/disputes/download https://uri.paypal.com/services/billing-agreements https://uri.paypal.com/services/checkout/payment-resources https://uri.paypal.com/services/payments/payouts https://api.paypal.com/v1/vault/credit-card/* https://uri.paypal.com/services/subscriptions",
3   "access_token": "A21AAJcvmX6rNY1sMU4_vLNkWHgo0xqU3RmJMeVFshNuE-RLJ5x_bk2zPYu1ac8m9RRK5n71RudJk1YDmw3Hfs83_2wuhyl1w",
4   "token_type": "Bearer",
5   "app_id": "APP-3X797586RY0965356",
6   "expires_in": 32400,
7   "nonce": "2026-06-11T14:49:59ZIRbtUoV2JwnsRKLwWuTrbfsIO5U00LYjUii_6FhaWUs"
8 }
```

6.4 Prueba exitosa — Crear orden (POST)

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** `https://api-m.sandbox.paypal.com/v2/checkout/orders`
- Auth Type:** Bearer Token
- Token:** A masked token is shown in a yellow box.
- Response:** 201 Created, 1.31 s, 1.61 KB
- Body (JSON):**

```
1 {
2   "id": "5G20918066888201F",
3   "status": "CREATED",
4   "links": [
5     {
6       "href": "https://api.sandbox.paypal.com/v2/checkout/orders/5G20918066888201F",
7       "rel": "self",
8       "method": "GET"
9     },
10    {
11      "href": "https://www.sandbox.paypal.com/checkoutnow?token=5G20918066888201F",
12      "rel": "approve",
13      "method": "GET"
14    },
15    {
16      "href": "https://api.sandbox.paypal.com/v2/checkout/orders/5G20918066888201F",
17      "rel": "update",
18      "method": "PATCH"
19    },
20    {
21      "href": "https://api.sandbox.paypal.com/v2/checkout/orders/5G20918066888201F/capture",
22      "rel": "capture",
23      "method": "POST"
24    }
25  ]
26 }
```

6.5 Prueba Exitosa — Consultar estado orden (GET)

GET <https://api-m.sandbox.paypal.com/v2/checkout/orders/5G20918066888201F> Send

Docs Params **Authorization** Headers 9 Body Scripts Settings Cookies

Auth Type
Bearer ...

Token
.....

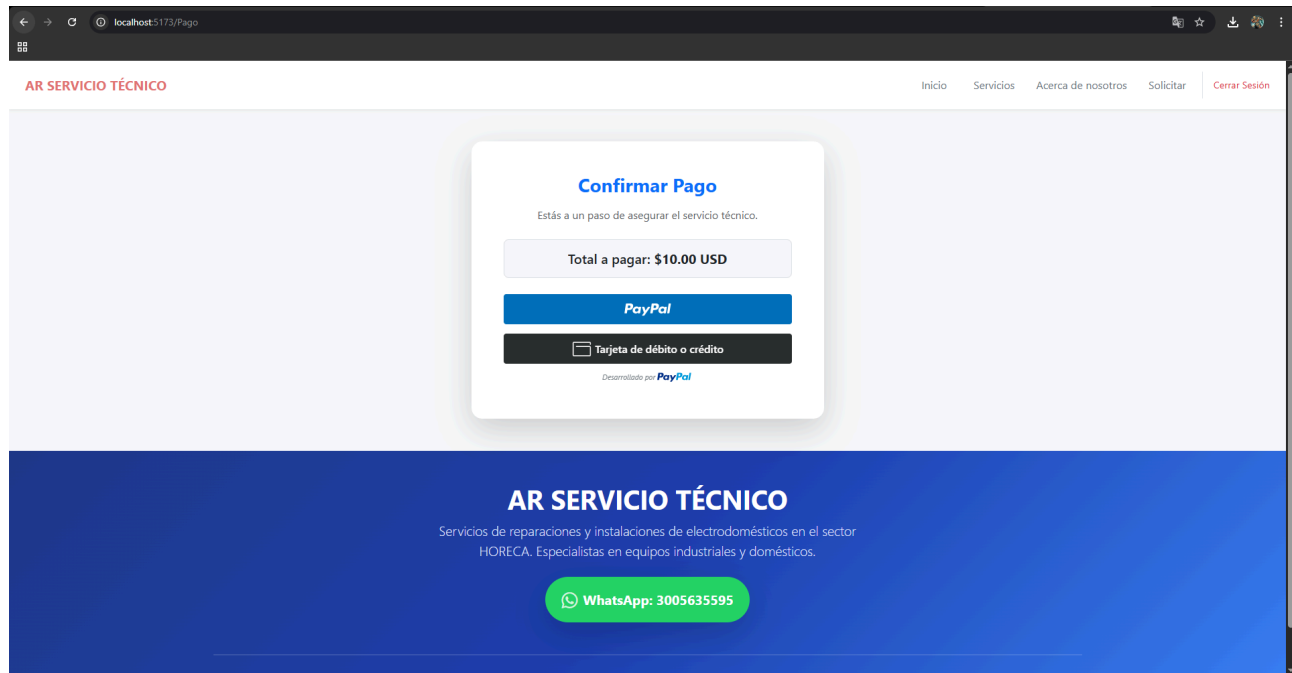
The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body Cookies 6 Headers 20 Test Results 200 OK • 1.25 s • 1.51 KB Save Response ...

{ } JSON Preview Visualize

```
1 {
2   "id": "5G20918066888201F",
3   "intent": "CAPTURE",
4   "status": "CREATED",
5   "purchase_units": [
6     {
7       "reference_id": "default",
8       "amount": {
9         "currency_code": "USD",
10        "value": "10.00"
11      },
12      "payee": {
13        "email_address": "sb-tnsob50965775@business.example.com",
14        "merchant_id": "A3RRVHQ8HVVY6Q"
15      }
16    }
17  ],
18  "create_time": "2026-05-11T14:52:19Z",
19  "links": [
20    {
21      "href": "https://api.sandbox.paypal.com/v2/checkout/orders/5G20918066888201F",
22      "rel": "self",
23      "method": "GET"
24    },
25    {
26      "href": "https://www.sandbox.paypal.com/checkoutnow?token=5G20918066888201F",
27      "rel": "approve",
28      "method": "GET"
29    },
30    {
31      "href": "https://api.sandbox.paypal.com/v2/checkout/orders/5G20918066888201F",
32      "rel": "update",
33      "method": "PATCH"
34    }
35  ]
36 }
```

6.6 Prueba Exitosa — Finalizar Pago(GET)





Pagar con PayPal

Con una cuenta de PayPal, reúne los requisitos para recompensas y la Protección al Comprador.

tatisnik1022@gmail.com

••••••••

[¿Ha olvidado su contraseña?](#)

Iniciar sesión

Abrir una cuenta

JD



\$10,00

Enviar a John Doe
Free Trade Zone, BOGOTA 110111, BOGOTA

[Modificar](#)

Pagar con



Saldo de PayPal

\$10,00

Preferida



Visa

****3532

[+ Agregar tarjeta de crédito](#)

[Continuar y revisar pedido](#)

prueba > Finalizar Pago/Captura

Save
Share
New Chat

POST
https://api-m.sandbox.paypal.com/v2/checkout/orders/5G20918066888201F/capture
Send

Docs
Params
Authorization
Headers 10
Body
Scripts
Settings

Auth Type
Bearer Token
Token
.....

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body
Cookies 6
Headers 20
Test Results

201 Created
1.97 s
2.7 KB
Save Response

JSON
Preview
Visualize

```

1  {
2    "id": "5620918066888201F",
3    "status": "COMPLETED",
4    "payment_source": {
5      "paypal": {
6        "email_address": "tatisnik1022@gmail.com",
7        "account_id": "FL5MBLSFUFBTW",
8        "account_status": "VERIFIED",
9        "name": {
10         "given_name": "John",
11         "surname": "Doe"
12       },
13       "address": {
14         "country_code": "CO"
15       }
16     },
17   },
18   "purchase_units": [
19     {
20       "reference_id": "default",
21       "shipping": {
22         "name": {
23           "full_name": "John Doe"
24         },
25         "address": {
26           "address_line_1": "Free Trade Zone",
27           "admin_area_2": "Bogota",
28           "admin_area_1": "Bogota",
29           "postal_code": "110111",
30           "country_code": "CO"
31         }
32       },
33       "payments": {

```

7. INTEGRACIÓN PROPUESTA

7.1 Flujo de notificación completo

#	Actor	Acción
1	Admin	
2	Backend	El endpoint
3		

....

de apoyo | Instructor: Ing. Edwin Marín

TALLER ADSO 2026 — xxxxxxxxxxxx xxxxx | Página 14